

Software Engineering Assignment

23K-0842, KINZA

Question 1: MediTrack System

1. Recommended Software Process Model

I recommend a Hybrid Software Process Model (Agile + Plan-Driven/Waterfall).

Justification (4 reasons):

1. Evolving Requirements → Agile Support

The system requirements are only partially known and will evolve through stakeholder feedback. Agile allows iterative development and continuous requirement refinement.

2. Strict Healthcare Regulations (HIPAA) → Plan-Driven Need

Due to sensitive medical data, strong documentation, validation, and compliance are required, which are better supported by a plan-driven approach.

3. Tight Timeline → Agile Advantage

The system requires a usable version within 4 months. Agile enables early delivery through incremental releases.

4. Suitable Team Size (12 members)

A team of 12 members fits well with Agile practices, promoting collaboration, communication, and faster iterations.

2. Hybrid Approach:

Plan-Driven Components:

- Authentication & Security
- Medical Records System
- Compliance features

Agile Components:

- UI/UX (dashboard)
- Appointment scheduling
- Messaging system
- Questionnaires

3. Design Concepts:

(a) Abstraction

Definition:

Abstraction means showing only essential features while hiding implementation details.

Example:

In the appointment scheduling feature, the user selects a doctor and time slot.

The system hides internal operations like database queries and scheduling algorithms.

(b) Information Hiding

Definition:

Information hiding restricts access to internal data and implementation details of a module.

Example:

The medical records module allows access only through secure APIs.

Other components cannot directly access or modify sensitive patient data.

4. Cohesion and Coupling:

Relationship:

- **Cohesion:** Measures how closely related functions within a module are
- **Coupling:** Measures how dependent modules are on each other

Good software design aims for high cohesion and low coupling

High Cohesion Example (Beneficial):

Appointment Management Module

- Handles booking, rescheduling, and cancellation
- All functionalities are related

Makes the module easier to maintain and understand

Low Coupling Example (Beneficial):

Messaging System and Medical Records System

- Messaging system interacts through APIs
- It does not directly access the medical database

Changes in one system do not affect the other.

Question 2: Smart Campus System

1. Recommended Architectural Pattern

I recommend a Microservices Architecture combined with MVC (Model-View-Controller).

Justification:

- **Scalability:** Each subsystem (Course Management, Security, etc.) can scale independently to support 55,000+ users
- **High Availability:** Failure in one service does not bring down the entire system
- **Real-time Processing:** Security subsystem can use streaming/microservices for camera data
- **Integration:** Easier to connect with legacy systems via APIs
- **Evolvability:** New features can be added without affecting existing services

2. Two Architectural Decisions for High Availability

Decision 1: Load Balancing with Redundant Servers

- Deploy multiple instances of each service
- Use a load balancer to distribute incoming requests

Benefit:

If one server fails, others continue serving users → no downtime

System Characteristics Addressed:

Availability + Performance

Decision 2: Database Replication and Failover

- Use primary and replica databases
- Automatic failover if the main database crashes

Benefit:

Ensures continuous data access and prevents system failure

System Characteristics Addressed:

Availability + Safety (data protection)

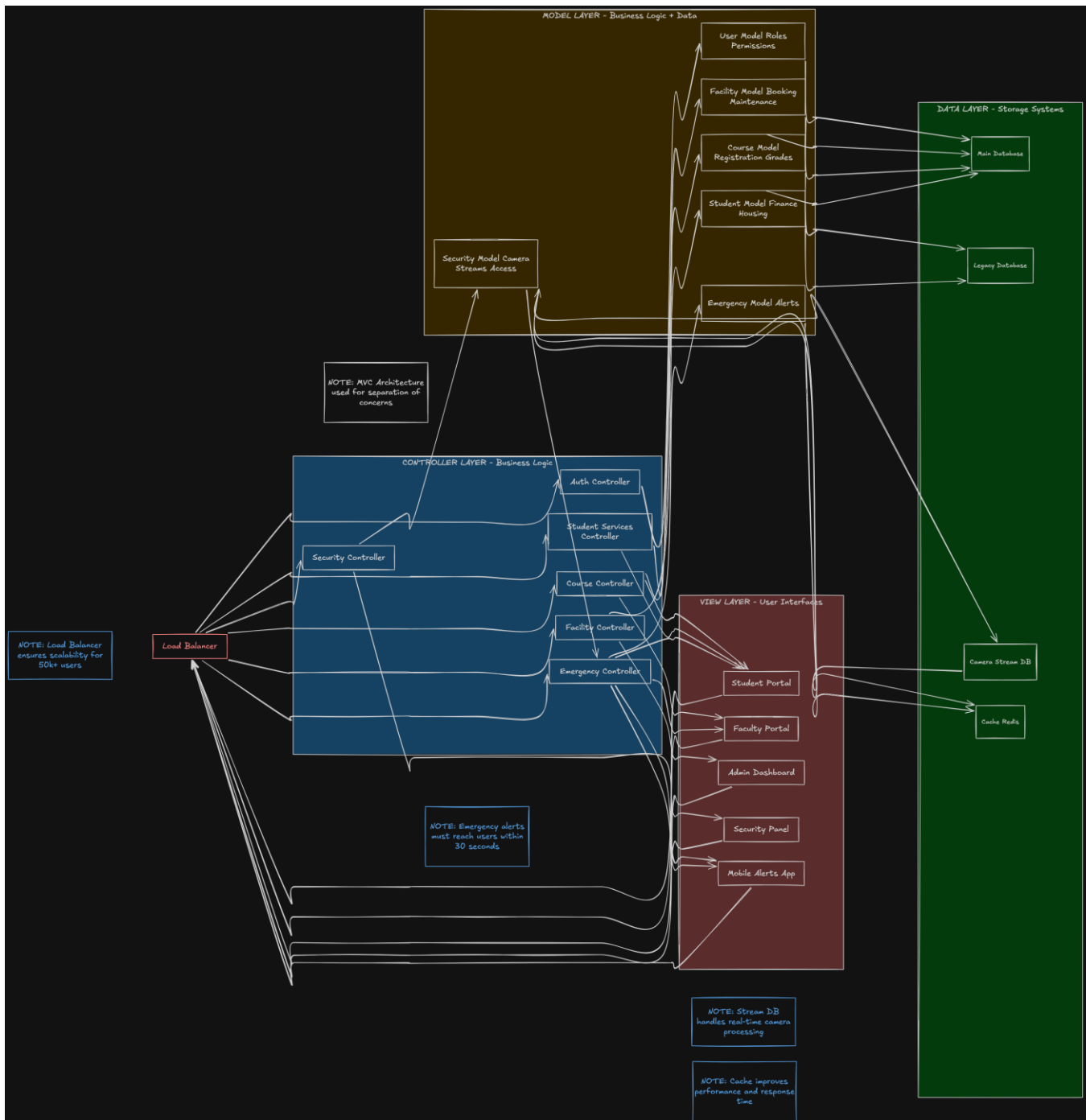
3. MVC Explanation:

View: Student, Faculty, Admin interfaces

Controller: Business logic & validation

Model: Databases and data handling

Flow: View → Controller → Model → Controller → View



4. Refactoring:

Refactoring is the process of improving the internal structure of code without changing its external behavior.

Example in Smart Campus System:

Initial Situation:

A single large module handles all user roles (students, faculty, admins).

Refactoring Action:

Split it into:

- StudentService
- FacultyService
- AdminService

Benefits:

- Better readability
- Easier maintenance
- Improved scalability

5. Importance of Modularity

Why Modularity is Important (10+ years lifespan):

The system will evolve continuously, so dividing it into independent modules is essential.

Benefits:

1. Easier Maintenance

- Bugs can be fixed in one module without affecting others

2. Independent Updates

- Example: Upgrade Security system without changing Course Management

3. Scalability

- High-demand modules (e.g., Emergency Notifications) can scale separately

4. Parallel Development

- Multiple teams can work on different modules simultaneously

How It Supports Maintainability:

- Clear separation of concerns
- Reduced system complexity
- Easier testing and debugging
- Faster feature addition over time

